

Zeva Connect — Mobile App

MVP Specification & Timeline — React Native (Expo)

4-5 Weeks to MVP	1 Solo Dev (AI-assisted)	Expo Framework	Shared API & Design System
---------------------	-----------------------------	-------------------	-------------------------------

1. Executive Summary

This document scopes the MVP for the **Zeva Connect mobile app**, built with React Native and Expo. The mobile app extends real-time clinic staff messaging to iOS and Android, giving staff the same chat experience they already have on the web — 1-to-1 and group messaging, attachments, and notifications — from their phones.

The mobile app is a new client built on top of the existing messaging API and real-time infrastructure. No new backend systems are required for the MVP — the mobile app consumes the same REST endpoints and Socket.io real-time layer already powering the web experience, and will carry over the same visual identity (colors, typography, and overall look and feel) for a consistent cross-platform experience.

2. Goals & Non-Goals

Goals for this MVP:

- Give clinic staff a native mobile experience for real-time messaging — 1-to-1 chats, group chats, and attachments.
- Support push notifications that work reliably even when the app is closed or the phone is locked.
- Reuse the existing single sign-on flow so staff don't manage a separate mobile password.
- Ship on both iOS and Android from a single codebase, using Expo's managed workflow to minimize native-code maintenance for a solo developer.

Explicitly out of scope for MVP:

- Voice/video calling.
- Offline message composition and sync queue (messages require an active connection to send in v1).
- Biometric app-lock (Face ID / fingerprint) — planned as a fast-follow, not MVP-blocking.
- Tablet-optimized / split-view layouts.

3. Feature Scope

Feature	Description
Authentication	Single sign-on from Zeva Clinic mobile session (or a dedicated mobile login screen using existing credential-based login), with automatic session refresh.
Conversation list	1-to-1 and group conversations, clinic-scoped staff directory, unread counts, online status.

Feature	Description
Chat window	Real-time messaging, typing indicators, read receipts, reactions, reply, edit, delete — matching web functionality.
Attachments	Send/receive images, videos, documents, and voice messages; native camera and gallery picker; in-app attachment viewer.
Push notifications	Native push notifications (via Expo Notifications) for new messages, working with the app backgrounded or closed.
Group management	Create groups, view members, admin controls (add/remove members, promote admin) — same permissions model as web.
Profile & settings	View own profile, notification preferences, theme (light/dark), logout.

4. Technical Approach

Stack

Layer	Choice	Why
Framework	React Native + Expo (managed workflow)	Fast solo-dev iteration, OTA updates, no native build maintenance for MVP scope
Navigation	React Navigation	Industry-standard stack/tab navigation for chat-list, chat, and settings flows
State management	Zustand + TanStack Query	Same pattern as the web client, for consistency and code-sharing of logic where possible
Real-time layer	Socket.io client	Connects to the same Socket.io server already running for the web app
Push notifications	Expo Notifications + backend push service	Device tokens registered against the existing push-subscription system, extended to support mobile push tokens alongside web push
Local storage	Expo SecureStore	Secure storage of access/refresh tokens on-device
Media handling	Expo ImagePicker, expo-av, expo-document-picker	Native camera, gallery, and file picker integration

Design consistency

The mobile app will carry over the same visual identity established on the web client — color palette, typography, iconography, and overall interaction patterns — adapted to native mobile UI conventions (bottom tab bars, native gestures, platform-appropriate modals) so the product feels familiar to staff moving between web and mobile, while still feeling like a proper native app rather than a wrapped webview.

5. Timeline (Solo Developer, AI-Assisted)

Estimated for a single developer working full-time, using AI-assisted development for boilerplate, component scaffolding, and debugging. Native mobile work (platform-specific push notification setup, permissions handling, device testing) is less accelerated by AI than typical web CRUD work, so this timeline is paced accordingly rather than optimistically compressed.

Phase	Scope
Week 1	Project setup (Expo + TypeScript), navigation structure, SSO/login flow, secure token storage, API client wired to existing backend.
Week 2	Conversation list + chat window UI, Socket.io real-time integration (messages, typing, read receipts), reactions/reply/edit/delete.
Week 3	Attachments (camera/gallery/document picker, upload flow), native attachment viewer, group creation and admin controls.
Week 4	Push notifications end-to-end (device token registration, background/killed-app delivery, tap-to-open-conversation), profile & settings screens, theming.
Week 5 (buffer)	Device testing across iOS/Android, bug fixes, app icon/splash screen polish, store-listing preparation.

Target: MVP feature-complete in 4 weeks, with Week 5 as buffer for device-testing and store-submission prep — giving a realistic 4-5 week window.

6. Distribution & Rollout

- Internal pilot distribution via Expo's over-the-air build sharing (Expo Go or internal distribution build) for early testing with a pilot clinic, before any app-store submission.
- Production release via TestFlight (iOS) and Google Play internal/closed testing track, ahead of a public store listing.
- OTA (over-the-air) updates via Expo Updates for JavaScript-level bug fixes and minor feature changes post-launch, without requiring a full app-store re-review for every small fix.

7. Risks & Considerations

- Push notification reliability varies by OS battery-optimization settings (especially on some Android OEM skins) — staff may need to be guided to disable battery optimization for the app for consistent delivery.
- App store review timelines (particularly iOS) are outside developer control and should be padded into any hard launch date.
- Camera/microphone/notification permissions must be requested with clear in-context explanations to avoid staff denying access on first prompt.

8. Recommendation

Proceed with a React Native (Expo) mobile app that consumes the existing messaging API and real-time infrastructure as-is, carrying forward the same design language as the web client. This keeps the mobile app light to build and maintain as a solo project, while giving clinic staff full-featured native messaging on their phones within a realistic 4-5 week timeline.